

Verifier-Guided Prompting for Intent-to-Configuration Translation: A Preregistered NetOps Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a paired confirmatory trial to test whether constrained prompting plus a deterministic verifier-guided generate→verify→repair loop (one allowed repair) increases deterministic-verifier semantic-correctness when translating operator natural-language intents into low-level device configurations. Design: N=150 synthetic intents sampled from a deterministic generator, shared_initial_candidate generation semantics, model = gpt-5-mini, deterministic validator = deterministic_netops_validator_v5, one initial generation per intent shared across baseline and guarded conditions, guarded pipeline permitted ≤1 repair call. Results (artifact-grounded): baseline = 149/150 (99.333%); guarded = 150/150 (100.0%); paired absolute difference = 0.006667 (≈0.67 percentage points); exact McNemar two-sided p = 1.0; 95% bootstrap percentile CI = [0.00, 0.02] (10,000 resamples, seed = 1729). Provider accounting: completed episodes = 300; model_calls_used = 151; repair-stage invocations = 1. Interpretation: on this preregistered synthetic corpus and under shared_initial_candidate semantics the guarded workflow produced a numerically negligible and statistically non-significant improvement over a near-ceiling baseline. We emphasize the methodological lesson that preregistered NetOps trials require baseline-calibration pilots to avoid ceiling effects that invalidate preregistered power assumptions. All execution and analysis artifacts are archived in the supplied execution bundle referenced by the execution manifest.

I. INTRODUCTION

Natural-language intent interfaces aim to reduce operator burden by letting humans express desired network changes as free text that is automatically translated into executable device configurations. Prior work documents that single-pass LLM outputs often contain syntactic errors, omissions, or semantic violations that can yield unsafe or unavailable states if applied directly [1][2][3]. A commonly proposed defense is a generate→verify→repair architecture in which an LLM produces a candidate, a deterministic verifier checks intent-level invariants or formalized constraints, and an automated repair (or human gate) corrects violations before execution [4][5]. Security studies of tool-augmented LLM agents further motivate deterministic gating because unchecked textual claims can become harmful actions in multi-tool workflows [6].

Research question. After an autonomous literature review and preregistration we posed a confined confirmatory question: does constrained prompting (structured templates with explicit semantic slots) combined with deterministic verifier-guided iterative feedback materially increase verifier-level semantic correctness of NL→configuration translation versus

an unconstrained baseline under a paired design? The trial (study_cand_2026_b2_v1) was preregistered and frozen before any model outputs were observed.

Contributions. (1) A fully preregistered, artifactized paired confirmatory trial measuring deterministic-verifier pass/fail on a programmatically generated synthetic corpus (N=150 paired intents). (2) Artifact-backed outcomes showing that, under shared_initial_candidate semantics and a one-repair budget, the guarded pipeline raised verifier pass rate from 149/150 to 150/150 (+0.67 pp), a numerically negligible and statistically non-significant improvement (exact McNemar p = 1.0; 95% bootstrap CI = [0.00, 0.02]). (3) A methodological recommendation that preregistered NetOps trials include baseline-calibration pilots and explicitly specify generation semantics (shared_initial_candidate vs independent sampling) because semantics materially change the estimand and interpretation.

II. RELATED WORK

Related literature falls into three strands; each strand provides partial motivation for the guarded generate→verify→repair architecture we evaluate and clarifies where prior claims and limitations remain.

Intent-to-configuration translation. A growing body of work explores converting operator natural-language intents into formal specifications or concrete device configurations. Empirical evaluations and surveys report that single-pass translation often attains only modest semantic-correctness on curated benchmarks, motivating post-generation checks and constrained prompting to improve fidelity [1][2]. Representative studies highlight varied task formulations (intent grammars, template-based translation, and intent-slot extraction) and report that success rates depend strongly on task granularity, the expressiveness of the target formalism, and prompt design choices [1]. Systematic reviews and targeted evaluations underscore that many NL→formal pipelines still produce semantic errors that formal verifiers can detect but that the end-to-end reliability required for unattended NetOps remains an open problem [2]. These findings justify using deterministic verifiers as the primary confirmatory endpoint and motivate task-bank designs that capture common operational idioms (reachability, ACL-like constraints, VLAN/MTU sequences, and uplink switchover patterns) rather than only isolated syntactic transforms.

Generate→verify→repair architectures and verifier technology. Several recent method papers and systems integrate deterministic or policy-guided verifiers with generation to improve correctness for infrastructure-as-code and configuration-repair tasks [4][7]. Approaches range from using verifier feedback for fine-tuning to runtime repair loops that iteratively correct a candidate until invariants hold. Reported improvements are heterogeneous: gains are larger when baseline model competence is modest, when larger repair budgets or multiple independent samples are allowed, or when verifier feedback provides actionable, fine-grained corrections rather than coarse pass/fail signals [4][7]. Critically, the effectiveness of verifier-guided repair depends on verifier coverage and the fidelity of the mapping layer that translates natural-language intents to verifier inputs; prior work emphasizes rigorous unit-testing of mapping code and staged integration with formal tools [5].

A complementary thread documents mature formal verification tools (e.g., NetKAT variants, KATch, Hoyan, and other symbolic verifiers) that can act as deterministic oracles for reachability, isolation, and policy checks in controlled topologies [5][9]. These tools vary in supported abstractions, latency, and scale: some are optimized for design-time proofs while others (e.g., recent NetKAT provers and scalable overlay checkers) demonstrate faster runtime checks for smaller topologies. Prior studies therefore treat verifier selection and verifier-coverage testing as design decisions that directly affect measured semantic-correctness and external validity.

Security, prompt-injection, and claim-to-action risks. Security- and attack-oriented analyses show that LLM-driven multi-tool workflows can convert false textual claims into concrete tool calls and harmful actions unless guarded by auditable gates or hardened planning primitives [6][8]. Empirical prompt-injection studies demonstrate practical exploitation strategies and optimization-based attacks against LLM-as-judge setups; corresponding defenses (prompt hardening, verifier-aware gating, and call-auditing) reduce risk but impose trade-offs with automation throughput [6][8]. These results motivate conservative guarded architectures that retain verifier traces and normalized configurations for operator review, a design choice we make explicit when framing operational interpretation beyond raw pass/fail rates.

How this study connects and differs. The present trial operationalizes a narrowly specified guarded architecture informed by the above strands: we evaluate verifier-level pass/fail on a deterministic synthetic corpus (so verifier selection and mapping tests are central), we limit repair budget to a single repair call to match a constrained operational budget, and we adopt `shared_initial_candidate` semantics to isolate the marginal effect of repair applied to an identical first-pass candidate. These design choices intentionally expose how sampling semantics, verifier coverage, and repair budgets interact with baseline competence—issues emphasized across the cited literature—and explain why previously reported larger guarded-vs-baseline gains can co-exist with a near-ceiling outcome in tightly controlled, verifier-capable synthetic settings [4][5][7].

III. METHODOLOGY

Preregistration and confirmatory design. The study was preregistered as `study_cand_2026_b2_v1` with a frozen analysis plan executed before any model outputs were observed. The primary estimand was the `paired_success_rate_difference_guarded_minus_baseline` evaluated with an exact two-sided McNemar test on deterministic-verifier pass/fail outcomes. The confirmatory sample specified $N=150$ distinct synthetic intents drawn deterministically from a preregistered generator, deterministic inclusion/exclusion rules (a rule-based ambiguity detector and verifier-coverage checks), a deterministic retry policy for provider timeouts (one retry), and a power target to detect an absolute +15 percentage-point improvement ($\approx 80\%$ power) under conservative discordant-pair assumptions. All elements of the preregistration (estimands, inclusion/exclusion rules, multiplicity plan, and analysis scripts) were frozen and archived prior to model calls.

Execution semantics: `shared_initial_candidate`, call budgets, and adapter enforcement. A central design decision was the execution-semantic enforcement of `shared_initial_candidate`: for each intent the execution adapter produced exactly one initial model generation and that identical sampled candidate was used in the paired evaluation of both baseline and guarded conditions. The guarded pipeline was permitted at most one additional repair call only if the deterministic verifier failed that initial candidate. Operationally, this means the baseline rate measures the validity of the single initial sampled candidate; the guarded vs baseline contrast therefore isolates the marginal effect of applying at most one repair to that identical candidate rather than measuring performance averaged over independent first-pass draws. The adapter enforced these constraints at runtime and recorded stage-level provider accounting; archived provider audits report `stage_counts = {shared_initial_generation: 150, repair: 1}` and `model_calls_used = 151`, consistent with the enforced one-initial plus at most-one-repair policy.

Model, sampling, and deterministic validator. Declared model: `gpt-5-mini` (provider record: `openai_responses`). The execution metadata include an explicit record that `model_configuration.json` declares `declared_model_version = "gpt-5-mini"` and `temperature = null` (unset). We emphasize the literal meaning: `temperature = null` in the recorded configuration is not equivalent to setting `temperature = 0` (deterministic decoding). Because no numeric temperature value or fixed random-seed was enforced in the preregistration or execution, model outputs may be nondeterministic across independent API sessions. This nondeterminism was anticipated in the preregistration and is reported explicitly here because sampling semantics materially affect estimand interpretation. Deterministic verifier: `deterministic_netops_validator_v5` (`validator_version = 5.0`) applied a `full_intent_constraint_validation` rule and returned for each candidate a boolean pass/fail verdict plus normalized final-configuration text and per-episode diagnostics (`parsed_command_count`, `required_command_count`,

observed_touched_field_count, violation_codes). These deterministic verdicts constituted the binary endpoint for the confirmatory test.

Task generator, corpus, prechecks, and mapping verification. Confirmatory tasks were produced deterministically by the preregistered generator (deterministic_netops_task_generator_v5) and were stratified across four intent families: reachability/isolation, ACL-like access changes, VLAN/MTU sequence changes, and uplink switchover patterns. Topologies were constrained to verifier-capable small networks (4–32 nodes) to ensure deterministic oracle coverage. Prior to batch execution we ran the preregistered mapping and verifier-coverage unit-tests: parser transformations that map LLM textual outputs to verifier inputs were unit-tested and passed. The preregistration specified deterministic exclusion rules (ambiguous intents routed to exploratory corpus; parse failures or unsupported verifier features excluded with deterministic reason codes); no confirmatory exclusions occurred in the archived run. Representative generator seeds, task_payloads, and expected_final_states are archived to support independent calibration or pilot reuse.

Scoring rules, primary analysis, and confidence procedures. The primary outcome was deterministic verifier pass (binary). Scoring used the archival scorer identifier deterministic_netops_validator_v5 and its full_intent_constraint_validation rule; for each episode the scorer produced a normalized_configuration string that was compared to the preregistered reference invariants for automated pass/fail determination. The confirmatory analysis used an exact two-sided McNemar test on the paired contingency (n_11, n_10, n_01, n_00) and computed a paired bootstrap percentile 95% confidence interval with 10,000 resamples (bootstrap_seed = 1729). The preregistered handling of incomplete pairs specified exclusion from the primary test (complete_pair_primary). Secondary contrasts (constrained vs baseline, guarded vs constrained) and multiplicity corrections were preregistered; under shared_initial_candidate enforcement these secondary contrasts were constrained by the adapter semantics and the available stage counts. All analysis code, contingency tables, and bootstrap parameters are archived and cited in the analysis artifacts.

Execution accounting, contamination controls, and reproducibility. Planned episodes were 300 (150 intents $\times$ 2 conditions); completed_episode_count = 300 and complete_pair_count = 150 in the archived manifest. Provider-call accounting and contamination checks were recorded automatically: completed_hosted_model_call_event_count = 151, failed_hosted_model_call_event_count = 0, cache_miss_count = 151, and contamination_summary.csv recorded zero flags. These runtime audits and per-call metadata (provider traces, shared_initial_response paths, and validator traces) were retained in the execution bundle to enable deterministic reconciliation of the paired counts and to support post hoc reproducibility checks. The preregistration also specified a contamination-detection checklist (session isolation, duplicate-

response checks, mapping unit-tests) that the adapter enforced prior to batch execution; all checks passed and are documented in the artifact logs.

IV. RESULTS

Execution and provider audit. The run completed as planned: completed_episode_count = 300 and complete_pair_count = 150. Provider-call accounting: model_calls_used = 151; completed_hosted_model_call_event_count = 151; failed_hosted_model_call_event_count = 0; cache_miss_count = 151. Stage-level counts: shared_initial_generation = 150 and repair = 1. No contamination flags or pre-execution unit-test failures occurred.

Primary numerical outcomes and contingency counts. The paired contingency counts (guarded $\times$ baseline) were: n_11 = 149 (both-pass), n_10 = 1 (guarded-only-pass), n_01 = 0 (baseline-only-pass), n_00 = 0 (both-fail). Per-condition success rates: baseline_success_count = 149/150 = 0.9933333333333333; guarded_success_count = 150/150 = 1.0. Estimated paired absolute difference (guarded - baseline) = 0.00666666666666671 ($\approx$ 0.67 percentage points). Exact McNemar two-sided p = 1.0. Paired bootstrap percentile 95% CI = [0.00, 0.02] (10,000 resamples; bootstrap_seed = 1729). These figures are reported verbatim from the archived analysis artifacts (analysis/results.json, analysis/paired_contingency_table.csv, analysis/condition_summary.csv).

Repair diagnostics and revision iterations. Only one guarded repair invocation occurred (1/150), consistent with the near-ceiling initial success rate. That single repair converted one initially invalid candidate to a valid configuration under the deterministic verifier. Median revision iterations for guarded episodes = 0 (IQR = 0). Validator traces include validation_before and validation_after states, normalized_configuration strings, parsed_command_count, required_command_count, violation_codes, and a valid flag for each episode; representative traces for tasks 000001–000006 are archived and demonstrate varied difficulty patterns while confirming validator behavior.

Representative artifact-grounded example. Example pair (task-000001, generator_seed = 1) intent: "Migrate edge1 to VLAN 30 and MTU 1600. For safety, shut edge1 down before changing either VLAN or MTU, then restore it to admin up. Do not modify any other interface." Shared-initial response and both condition responses returned the expected four-command sequence. Validator diagnostics for task-000001: parsed_command_count = 4, required_command_count = 4, observed_touched_field_count = 3, and valid = true. The shared-initial candidate SHA-256 and normalized_configuration strings are archived in the execution bundle as part of the scorer traces.

Sensitivity and missingness. The preregistered sensitivity analysis treating missing guarded outputs as failures was not invoked because failed_hosted_model_call_event_count = 0

and unscorable episodes = 0. No exclusions or parse failures occurred and `contamination_summary.csv` is empty.

Compact evidence tables. Table 1 (paired contingency) and Table 2 (execution audit summary) summarize the confirmatory counts and provider accounting; both tables are artifact-grounded and constructed from `analysis/paired_contingency_table.csv` and `analysis/condition_summary.csv`.

Interpretation of numeric outcome. On this synthetic corpus and under `shared_initial_candidate` semantics the guarded workflow produced a numerically negligible absolute improvement (+0.67 pp) from an already near-ceiling baseline. The exact McNemar $p = 1.0$ and the bootstrap CI containing zero indicate no statistically supported confirmatory benefit under the preregistered estimand. Importantly, the observed baseline success rate ($\approx 99.33\%$) violated the preregistered power assumption (target detectable effect = 15 pp), producing a ceiling effect that invalidated the originally planned sensitivity.

V. DISCUSSION

Ceiling effect and implications for preregistration. The preregistered power calculation assumed a substantially lower baseline; observed `baseline_success_rate = 0.9933` left almost no headroom for the guarded repair to show a large absolute improvement. This ceiling effect invalidated the preregistered detectable-effect assumptions (target = 15 percentage points) and reduced the trial’s ability to demonstrate benefit. We therefore recommend that preregistered NetOps trials include a short baseline-calibration pilot (e.g., 20–40 tasks sampled from the planned generator and prompt style) to estimate baseline competence and adjust N or task difficulty before committing to confirmatory sampling. The archived artifacts contain the deterministic task generator seeds and representative pilot-able tasks for such calibration.

Semantics-driven estimand differences and practitioner implications. Under `shared_initial_candidate` semantics the baseline measures the single initial sampled candidate while the guarded condition quantifies the marginal value of at most one repair applied to that same candidate. Independent-sampling designs or multiple-repair budgets yield different operational estimands—expected performance averaged over independent draws or repair-driven robustness gains—and can produce larger observable improvements when baseline sampling variance is larger. Practitioners should choose sampling semantics that align with operational constraints (e.g., whether the deployed system will re-sample across alternatives or apply repairs to a fixed first-pass candidate).

Operational affordances beyond the binary pass/fail metric. Deterministic guarded workflows provide operational benefits not captured by a single binary endpoint: auditable normalized configurations for operator review, concise diagnostic traces (`parsed_command_count`, `violation_codes`) that accelerate triage, and deterministic verifier verdicts suitable for policy gating. Even when marginal verifier-pass improvements are small on constrained corpora, these operational affordances can justify guarded architectures in practice; the archived

validator traces demonstrate the concrete diagnostic strings that would be available to operators.

Threats to validity. Internal validity: adapter-enforced call budgets and provider audit mitigate cross-condition leakage; the `shared_initial_candidate` semantics intentionally reduce sampling variance but do not capture per-condition randomness. Construct validity: deterministic validator pass/fail is a proxy for semantic correctness and depends on mapping code and verifier coverage; mapping risks were mitigated with unit-tests and archived artifacts but residual risk remains. External validity: experiments used a single declared model (gpt-5-mini), synthetic tasks, and small topologies (4–32 nodes); claims are limited to the archived corpus. Conclusion validity: preregistered power assumptions were invalidated by the high baseline, constraining confirmatory inference. All such limitations are reported and linked to archived artifacts and counts.

Recommendations and future experimental designs. Based on the artifact-grounded outcomes we recommend: (1) baseline-calibration pilots to select N and difficulty; (2) explicit preregistration of generation semantics (`shared_initial_candidate` vs independent) and repair budgets; (3) repair-budget arms (multiple allowed repairs) to estimate diminishing returns; (4) model-diversity arms to assess generality across model families and sampling parameters; and (5) incremental scaling of verifier coverage and topology size with unit-tested mapping code before batch execution. The execution manifest and representative task set provide seeds and code to implement these next-step designs reproducibly.

Reproducibility and artifact availability. All code, task-generator seeds, model-configuration metadata, raw model-call logs, verifier inputs/verdicts, and analysis artifacts (contingency tables, bootstrap parameters, and provider-call audits) are archived in the execution bundle referenced by the execution manifest. Key numeric analysis parameters are recorded (`bootstrap_seed = 1729`; `bootstrap_resamples = 10000`), and all analysis CSVs and validator traces cited above are available for independent verification. The archived `initial_master_prompt` reference and preregistration record are included as immutable artifacts.

VI. LIMITATIONS

- Single-model constraint: experiments used only gpt-5-mini; results do not generalize across models or sizes.
- Synthetic corpus and scale: tasks were programmatically generated and limited to verifier-capable toy-to-small topologies (4–32 nodes); external validity to production WANs and heterogeneous vendor CLIs is untested.
- `Shared_initial_candidate` semantics and execution contract limited repair budget to at most one guarded repair call per intent; different generation semantics or multiple-repair budgets may yield different outcomes.
- Ceiling effect and preregistration mismatch: baseline correctness was far higher than the pilot assumptions used for power calculations (planned detectable effect

= 15 percentage points), reducing the trial’s ability to demonstrate benefit and illustrating sensitivity to baseline calibration.

- Verifier coverage and specification translation: the study measures deterministic validator pass/fail on the preregistered invariants but does not claim safety guarantees beyond the deterministic validator’s modeled properties.
- Security and adversarial robustness: the experiment did not evaluate adversarial prompt-injection or adversary-driven intents; separate targeted studies are required to assess claim-to-action vulnerabilities in agentic NetOps workflows.

VII. CONCLUSION

We executed a fully preregistered paired confirmatory trial comparing baseline free-text prompting with constrained-template prompting plus a single verifier-guided repair under `shared_initial_candidate` semantics. On a deterministic synthetic corpus with `gpt-5-mini` and `deterministic_netops_validator_v5`, the guarded pipeline increased verifier pass rate from 149/150 to 150/150 (≈ 0.67 pp), a numerically tiny and statistically non-significant difference (exact McNemar $p = 1.0$; 95% bootstrap CI = [0.00, 0.02]). The principal scientific lesson is methodological: preregistered NetOps trials should include baseline calibration to avoid ceiling effects and preserve statistical power. Technically, our artifacts bound the marginal benefit of a one-repair guarded pipeline under `shared_initial_candidate` semantics; evaluating independent-sampling semantics, larger repair budgets, model diversity, and production-scale topologies remains important future work.

DISCLOSURE STATEMENT

This study was executed autonomously after an autonomy lock under the archived master prompt (provenance/master_prompt.txt, SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). Preregistration identifier: `study_cand_2026_b2_v1` (preregistration was frozen prior to observing outcomes and the preregistration record is archived). Declared model/tooling: declared model = `gpt-5-mini` (provider record: `openai_responses`); execution adapter family = `hosted_netops_gvr_v1`; deterministic validator = `deterministic_netops_validator_v5` (validator_version = 5.0). Runtime sampling semantics (explicit): the archived `model_configuration.json` records temperature = null (unset); because no numeric temperature or fixed random-seed was enforced, model outputs may be nondeterministic across independent API sessions. Autonomy and human role: a human Research Director selected the topic family and constrained the initial master prompt prior to the autonomy lock; after the lock the autonomous system performed literature review, study selection, preregistration, code generation, experiment execution, analysis, manuscript drafting, autonomous peer review, and revision. No human manually edited technical content, analyses, figures, captions,

or references after the autonomy lock. Key execution facts reported in the paper (artifact-grounded): completed episodes = 300; complete paired tasks = 150; `model_calls_used` = 151; repair-stage invocations = 1. All runtime sampling metadata, provider-call traces, verifier inputs/verdicts, and analysis artifacts are archived in the execution bundle referenced by the execution manifest.

REFERENCES

- [1] <https://doi.org/10.1002/nem.2313>
- [2] <https://doi.org/10.1145/3786583.3786898>
- [3] <https://arxiv.org/abs/2604.22513>
- [4] <https://doi.org/10.1145/3605764.3623985>
- [5] <https://doi.org/10.1145/3656454>
- [6] <https://doi.org/10.1002/widm.70111>